ICS 2105 Data Structures & Algorithms

Prof Muliaro Wafula

Outline

- **Data Structures -** Definition, Classification of Data Structures, Operations on Data Structures, Abstract Data Type (ADT), Preliminaries of algorithms. Time and Space complexity.
- **Searching -** Linear search, Binary search, Fibonacci search.
- **Sorting**- Insertion sort, Selection sort, Exchange (Bubble sort, quick sort), distribution (radix sort), merging (Merge sort) algorithms

- Data Structures is the concept of set of algorithms used to structure the information.

➢ These algorithms are implemented using C, C++, Java, etc

- ➢ Structure the information means store and process data in an efficient manner.
- ➢ To store and process data we may use the following operations
- 1.create() 6.sorting()
- 2.insert() 7.merging()
- 3.delete() 8.splitting()
- 4.display() 9.traversal()
- 5.searching()

➢ So data structure may contain algorithms, use for different operations implement these algorithms by a programming language

➢ For example for stack data structure write algorithms for different operations

- 1.push,
- 2.pop and
- 3.display.

Classification of Data Structures

- **Primitive data structures:**
- ➢ Primitive data structures are built in types in most programming languages. They are
- ➢ Integer: It is whole numbers. i.e. negative values,0,positive values
- ➢ Boolean: it represents true or false.
- Float
- Character

- **Non-primitive data structures**
- ➢ They are Array, Structure, Union, Files etc
- ➢ A Non-primitive data type is further divided into Linear and Non-Linear data structure

(ii)Non-primitive data structures:

- These are derived from primitive data structures.
- They are Array, Structure, Union, Files etc
- A Non-primitive data type is further divided into Linear and Non-Linear data structure.

(a)Linear data structures:

- Here the data elements are connected in a sequence manner.
- Examples are Arrays, Linked List, Stacks and Queues.

Array:

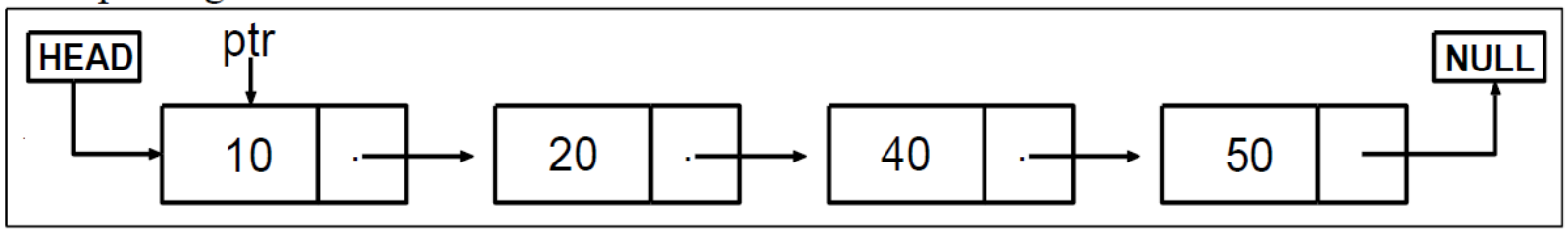
- It is collection of elements of the same type

	0	1	2	3	4
a[5]=	10	20	30	40	50

Linked List:

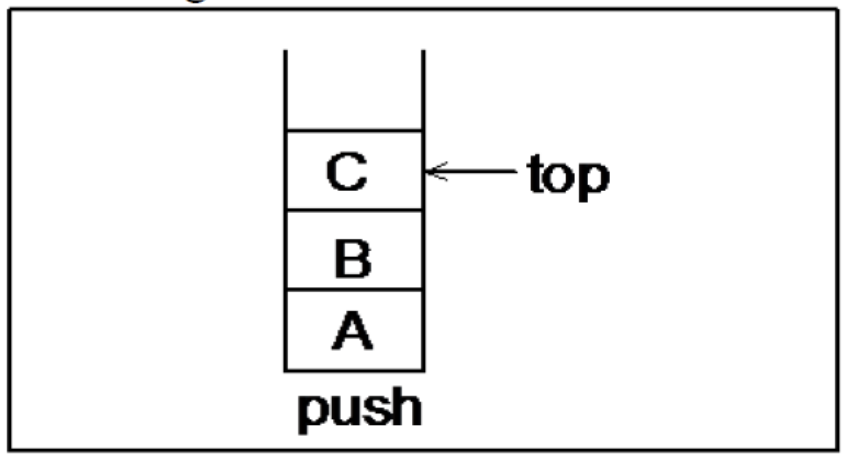
- *linked list* or single linked list is a sequence of elements in which every element has *link* to its next element in the sequence.
- Every element is called as a "node". Every "node" contains two fields, *data* and link. The data is a value or string and link is an address of next node.

- The first node is called HEAD which is an empty node contains an address of the first node so it link to the first node.
- The first node link to the second node and so on.
- The last node does not link to address but link to NULL. Let ptr be a pointer to the linked list. The example is given below



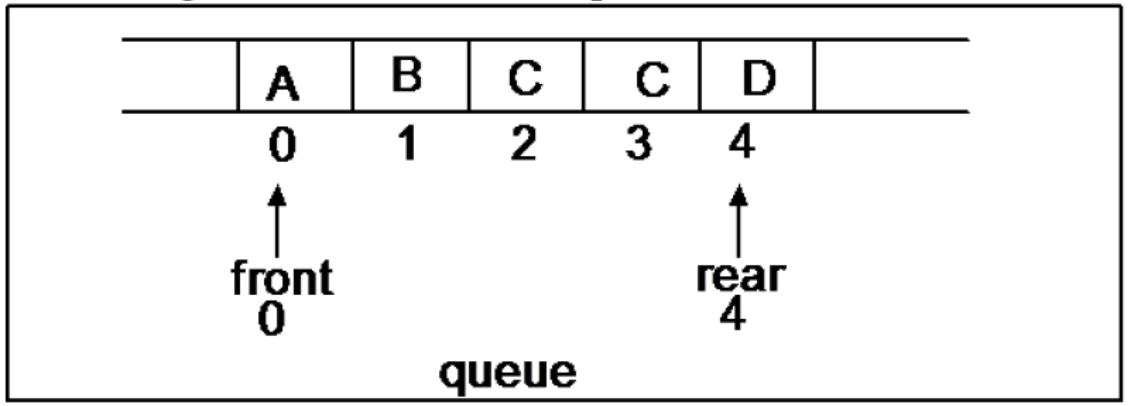
Stack:

- A stack is a data structure in which additions and deletions are made at the top of the stack. So we can perform two operations on stack.
  1. Adding elements into the stack known as push;
  2. Deleting elements from the stack known as pop



Queue:

- A queue is a data structure in which additions are made at one end and deletions are made at the other end. We can represent a queue in an array.
- Here we can perform two operations on queue.
  1. Adding elements into the queue known as insertion at rear
  2. Deleting elements from the queue known as deletion from front

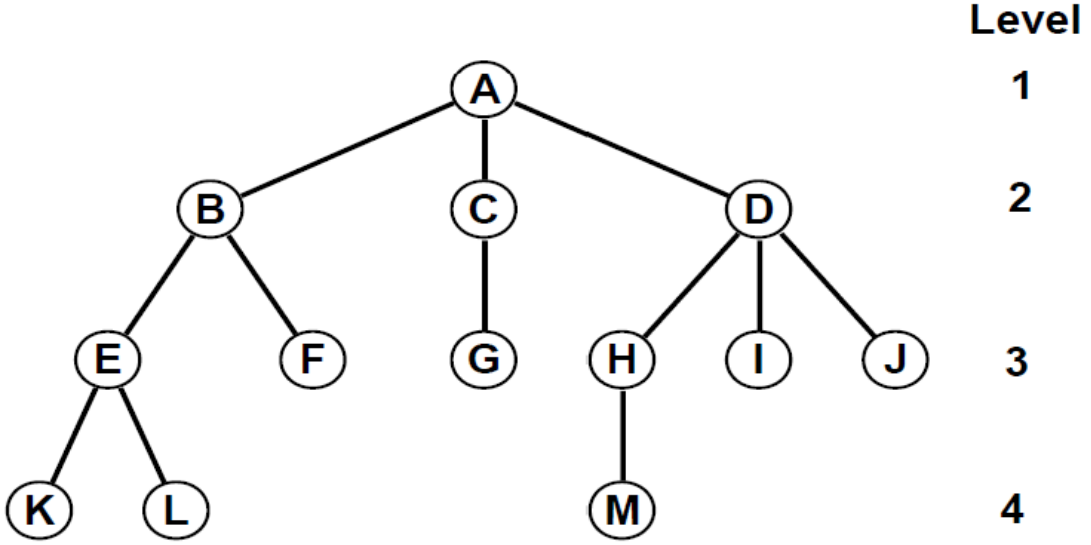


(b)Non-linear data structures:

- Here data elements are not connected in a sequence manner.
- Examples are: Trees and Graphs.

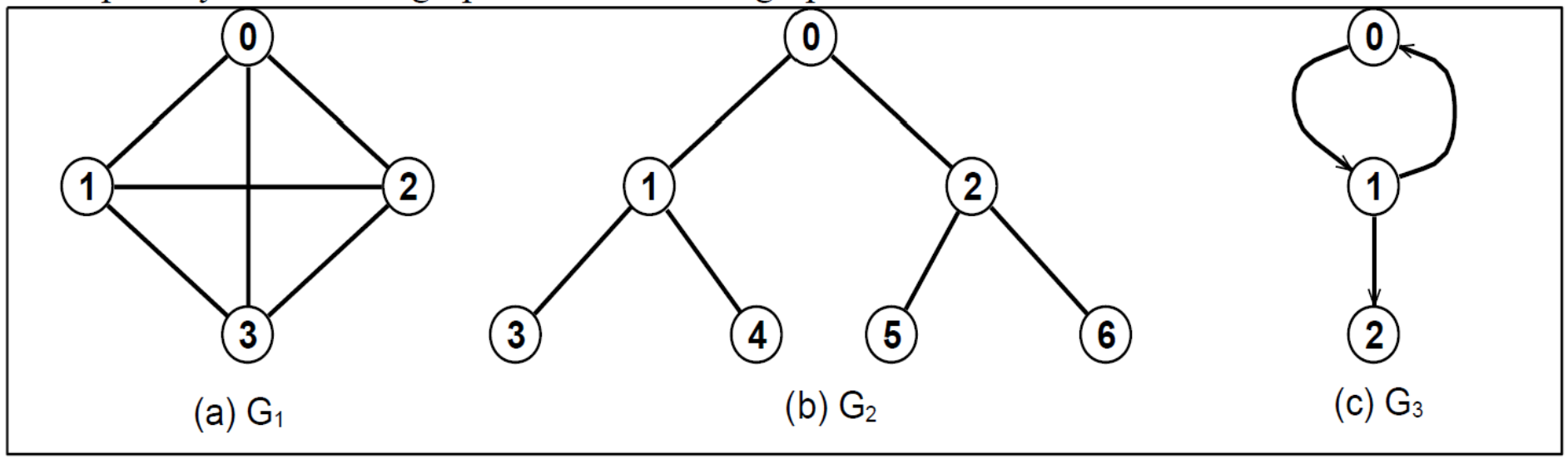
Tree:

- The tree is defined as a finite set of one or more nodes such that
  1. One node is called a root node and
  2. Remaining nodes partitioned into sub trees of the root.



Graph:

- A graph is a pictorial representation of a set of points or nodes termed as **vertices**, and the links that connect the vertices are called **edges**.
- A Graph(G) consists of two sets V and E where V is called vertices and E is called edges. We also write G = (V,E) to represent a graph.
- A Graph may be directed graph and undirected graph.



- The Fig(a),Fig(b) are called undirected graph & Fig(c) is called directed graph.

Differences between Linear and Non Linear Data Structures:

Linear Data Structure	Non-Linear Data Structure
Every data element is connected to its previous & next one	Every data element is connected with many other data elements.
Data is arranged in a sequence manner	Data is not arranged in a sequence manner
Data can be traversed in a single run	Data cannot be traversed in a single run
Ex: Array, Stack, Queue, Linked List	Ex: Tree, Graph
Implementation is easy	Implementation is difficult

(3) Operations on Data Structures:

- The different operations used on data structure are:

1.Create:

- Here we reserve memory for program elements: This can be done by using malloc or calloc function. We can create a data structure with giving different elements.

2.Insert:

- Here we reserve memory for program element. This can be done by using malloc or calloc function. We can insert an element into a data structure.

3.Delete:

- It delete memory space allocated for specified data structure using free().

4.Display:

- It deals with accessing a particular data within a data structure.

5.Searching:

- It finds the data item in the list of data items.
- It also find the location of all elements.

6.Sorting:

- It is the process of arranging all data items in a data structure in a particular order say either in ascending order or in descending order.

7.Merging:

- It is the process of combining the data items of two different sorted list into a single sorted list.

8.Splitting:

- It is the process of partitioning single list to multiple list.

9.Traversal:

- It is the process of visiting each and every node of a list in systematic manner.

Abstract Data Type(ADT):

- An abstract data type (ADT) is a collection of values and a set of operations without specifying its implementation.
- For example in Array ADT set of values are index and item & set of operations are create(),retrieval() and store().
- The purpose of the ADT is to hide the implementation details of a data structure, thus improving software maintenance, reuse and portability.
- The developers of ADT will adapt changing requirements and save time.
- The users of ADT are concerned with the interface, but not the implementation.
- The different ADTs given by

  String ADT, List ADT, Stack (last-in, first-out) ADT,
  Queue (first-in, first-out) ADT
  Binary Search Tree ADT etc

Example:

- A List ADT contains operations known as add element, remove element, etc.
- A List ADT can be represented by an array-based implementation or a linked-list based implementation. In this the linked-list based implementation is so commonly used.
- Similarly, a Binary Search Tree ADT can be represented in different ways with the same operations known as insert, remove, display, etc.